

AMRITA VISHWA VIDYAPEETHAM

Data Structure and Algorithm

LAB MANUAL

Submitted to	
Name	Dr. Rajkumar Batchu
Department	School of computing
Designation	Assistant Professor
Submitted by	
Name	M. Gokul krishna
Roll No	AV.SC.U4CSE25126
Year/Sem/Section	2nd YEAR/SEM-3/CSE-B

S.no	programs	Pg.no	remark
1	**a)** A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown from n to 1, followed by "Launch!". **b)** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate (power), where P = principal growth factor, n = number of years. **c)** An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **d)** In a logistics company, the number of possible ways to arrange n different packages is calculated using the factorial formula . Develop a recursive program to calculate the factorial of a given number. **e)** Write a recursive Python program to print the first n terms of Fibonacci series.	5-11	
2	a)write a python program to perform linear search on the given input b)Write a python program to perform binary search on the given input. Verify by taking sorted array and unsorted array as a list.	12-18	
3	A) write a python program to implement bubble sort, where the elements are taken from user. B) write a python program to implement selection sort, where the elements are taken from user. C) write a python program to implement insertion sort, where the elements are taken from user.	19-24	
4	A) write a python program to implement Merge sort, where the elements are taken from user. B) write a python program to implement Quick sort, where the elements are taken from user.	25-29	

5	A) Write a program to implement SLL with the below operations. Create linked list, Insert Beginning, Insert End, Insert at Index, delete by value, delete first node, delete last node, count number of nodes, display/transverse exit. B) Write a program to implement DLL with the below operations. Create linked list, Insert Beginning, Insert End, Insert at Index, delete by value, delete first node, delete last node, count number of nodes, display/transverse exit.	30-36	
6	A)Write a python program to implement stack operations such as push, pop, peek, display using an array and a linked list.	37-45	
7	A)write a python program to implement a queue and its operations such as enque,dequeue , peek and display with a menu driven approach implement using array and linked list data structure B) A)write a python program to implement a Circula queue and its operations such as enque,dequeue , peek and display with a menu driven approach implement using array and linked list data structure	46-63	
8	write a python program to implement a binary tree with its operations using a array and a linked list	64-72	
9	write a python program to implement a binary search treee with its operation and traversals	73-79	
10	1.0)Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing)	80-85	

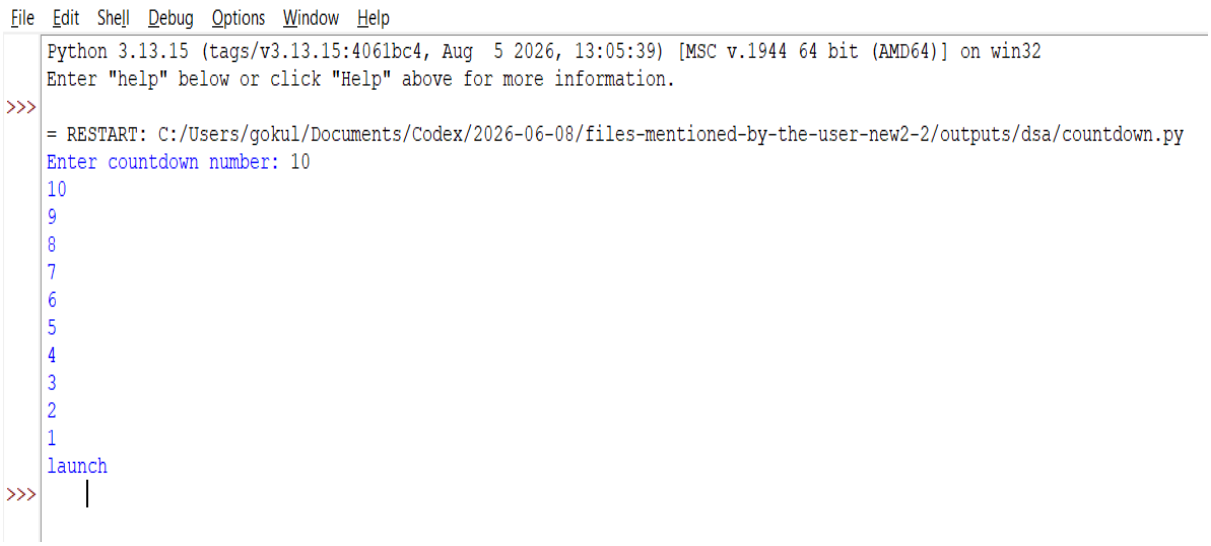
Week 1

a) A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown from n to 1, followed by "Launch!".

Program:

```
def count(n):
    if n == 0:
        print("launch")
        return
    print(n)
    count(n-1)
n= int(input("Enter countdown number: "))
count(n)
```

output:



Explanation:

- Best Case: O(n)
- Worst Case: O(n)
- Time Complexity: O(n)
- Space Complexity: O(n)
- Reason: The function is called n times, so the time complexity is O(n). Since all recursive calls are stored in the call stack until the base case is reached, the space complexity is O(n).

Error Table:

s.no	Error obtained	Rectification
1.	Infinite loop	Iteration at correct place
2.	Calling function at wrong place	Placed with correct syntax

b) Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate (power), where

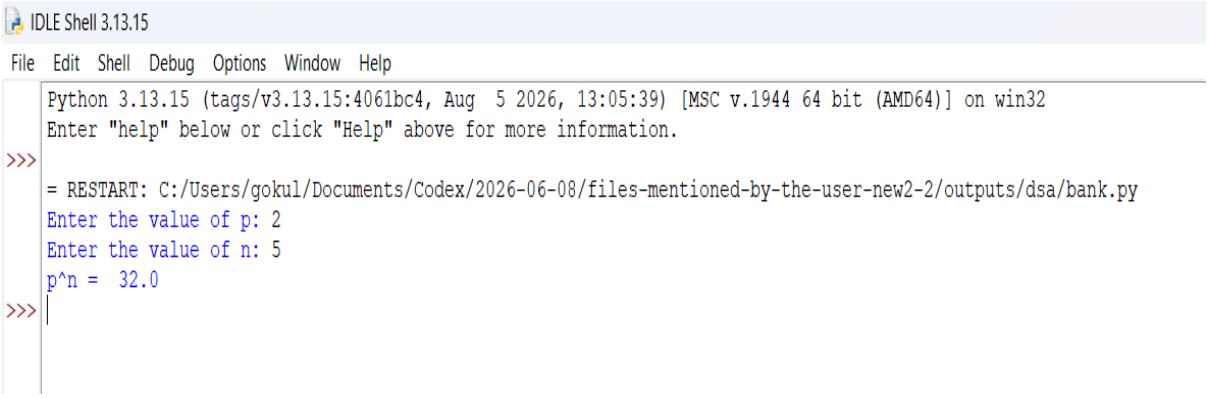
P = principal growth factor,

n = number of years.

Program:

```
def power(p,n):
    if n == 0:
        return 1
    return p*power(p,n-1)
p = float(input("Enter the value of p: "))
n = int(input("Enter the value of n: "))
print("p^n = ",power(p,n))
```

Output:



Explanation:

- **Best Case:** O(1)
- **Worst Case:** O(n)
- **Time Complexity:** O(n)
- **Space Complexity:** O(n)

Reason: The best case occurs when n = 0, because the function immediately returns 1, giving O(1) time and space. In the worst case, the function makes n recursive calls, resulting in O(n) time. The recursive calls are stored in the call stack, resulting in O(n) space.

Error Table:

s.no	Error obtained	Rectification
1.	Return 0	Return 1
2.	Indentation error	Corrected using proper induntation

c) An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

Program:

```
def search_employee(ids, target, index):
    if index == len(ids):
        return False

    if ids[index] == target:
        return True

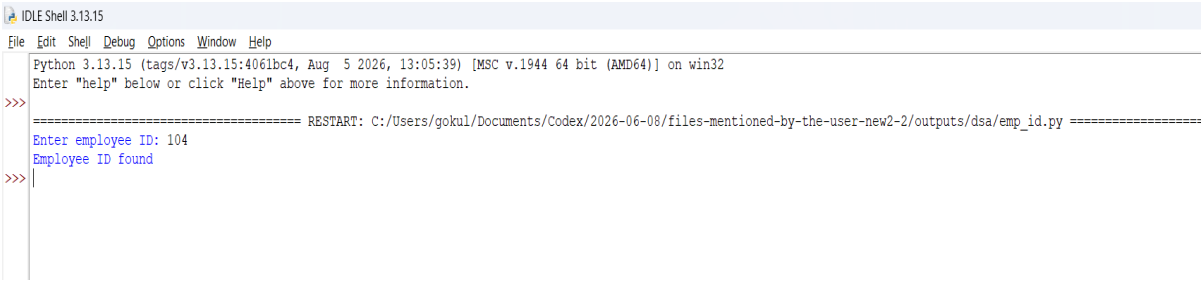
    return search_employee(ids, target, index + 1)


ids = [101, 102, 103, 104, 105]

target = int(input("Enter employee ID: "))

if search_employee(ids, target, 0):
    print("Employee ID found")
else:
    print("Employee ID not found")
```

output:



Explanation:

- **Best Case:** O(1)
- **Worst Case:** O(n)
- **Time Complexity:** O(n)
- **Space Complexity:** O(n)

Reason: The best case occurs when the required employee ID is the first element of the list, so only one comparison is needed, giving O(1) time and space. The worst case occurs when the employee ID is the last element or is not present in the list. In this case, all n elements are checked, resulting in O(n) time. Since recursive calls are stored in the call stack, the worst-case space complexity is O(n).

Error Table:

s.no	Error obtained	Rectification
1.	Forgot to declare n<=1	rectified
2.	Formula mistake	rectified

d) In a logistics company, the number of possible ways to arrange n different packages is calculated using the factorial formula . Develop a recursive program to calculate the factorial of a given number.

Program:

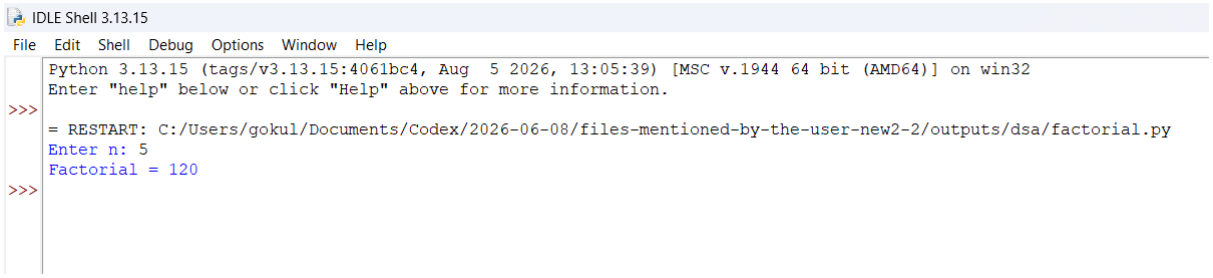
```
def factorial(n):
    if n == 0 or n == 1:
        return 1

    return n * factorial(n - 1)


n = int(input("Enter n: "))
```

```
print("Factorial =", factorial(n))
```

output:



Explanation:

- **Best Case:** O(1)
- **Worst Case:** O(n)
- **Time Complexity:** O(n)
- **Space Complexity:** O(n)

Reason: The best case occurs when n is 0 or 1, because the function immediately returns 1, resulting in O(1) time and space. In the worst case, the function makes recursive calls from n down to 1, resulting in O(n) time. The recursive calls occupy the call stack, so the space complexity is O(n).

Error Table:

s.no	Error obtained	Rectification
1.	Emplist	Emp_list
2.	print	Return

e) Write a recursive Python program to print the first n terms of Fibonacci series.

Program:

```
def fibonacci(n):
    if n == 0:
        return 0

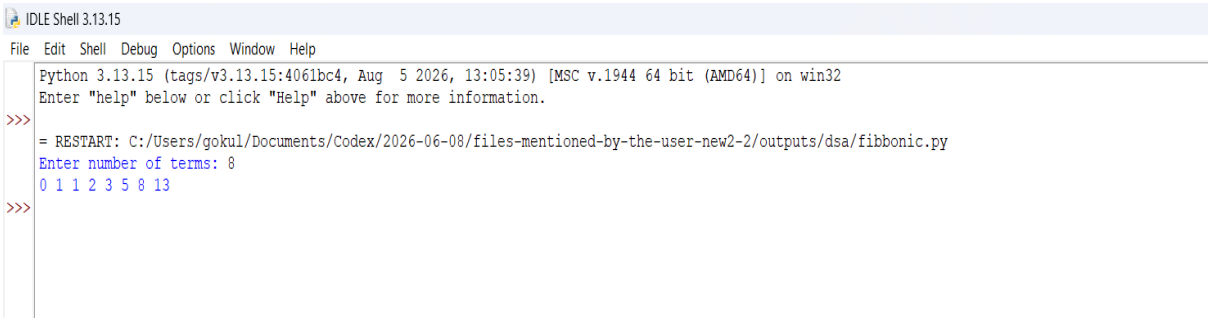
    if n == 1:
        return 1

    return fibonacci(n - 1) + fibonacci(n - 2)


n = int(input("Enter number of terms: "))

for i in range(n):
    print(fibonacci(i), end=" ")
```

output:



Explanation:

- **Best Case:** O(1)
- **Worst Case:** $O(2^n)$
- **Time Complexity:** $O(2^n)$
- **Space Complexity:** O(n)

Reason: The best case occurs when n is 0 or 1, because the function immediately returns the result, giving O(1) time and space. In the worst case, every recursive call creates two more recursive calls, causing the number of calls to grow exponentially. Therefore, the time

complexity is $O(2^n)$. Although there are many recursive calls, the maximum depth of the call stack is n, so the space complexity is O(n).

s.no	Error obtained	Rectification
1.	n+1	n-1
2.	Syntax error	Missed "(" at the definig the function

Week-2:

a)Write a program to implement Linear Search.

Program:

```
# Taking the number of elements
n = int(input("Enter number of elements: "))

# Taking array elements from the user
arr = []

for i in range(n):
    value = int(input("Enter element: "))
    arr.append(value)

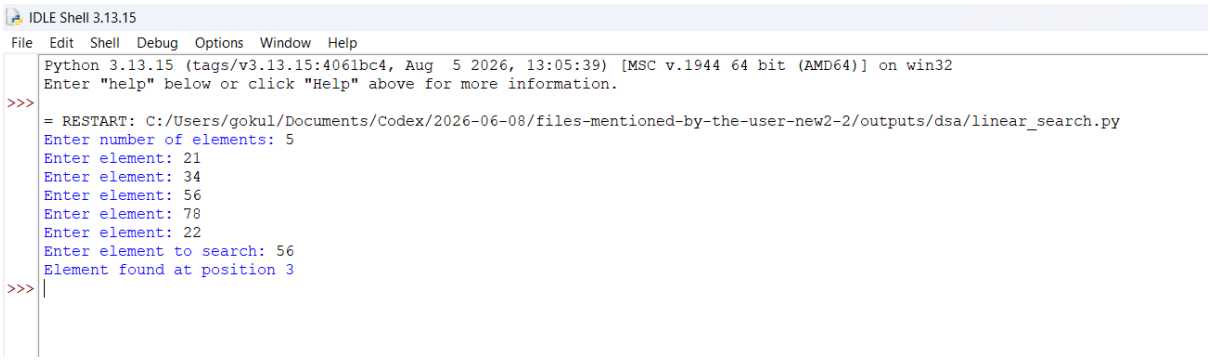
# Taking the element to search
target = int(input("Enter element to search: "))

# Linear search
found = False

for i in range(n):
    if arr[i] == target:
        print("Element found at position", i + 1)
        found = True
        break

if not found:
    print("Element not found")
```

Output:



Explanation:

- **Best Case:** O(1)
- **Worst Case:** O(n)
- **Time Complexity:** O(n)
- **Space Complexity:** O(n) for storing the input array

Reason: In the best case, the required element is the first element, so only one comparison is required. In the worst case, the element is at the last position or is not present, so all n elements must be checked. Therefore, the worst-case time complexity is O(n).

Error Table:

s.no	Error obtained	Rectification
1.	Return placed in wrong place	Corrected later
2.	Calling the function	Rectified

b) Write a program to implement Binary Search with sorted array as an input.

Program:

```
# Binary Search on a sorted array
# Taking the number of elements
n = int(input("Enter number of elements: "))

# Taking sorted array as input
arr = []

for i in range(n):
    value = int(input("Enter element: "))
    arr.append(value)

# Taking the element to search
target = int(input("Enter element to search: "))

# Binary search
low = 0
high = n - 1
found = False

while low <= high:
    mid = (low + high) // 2

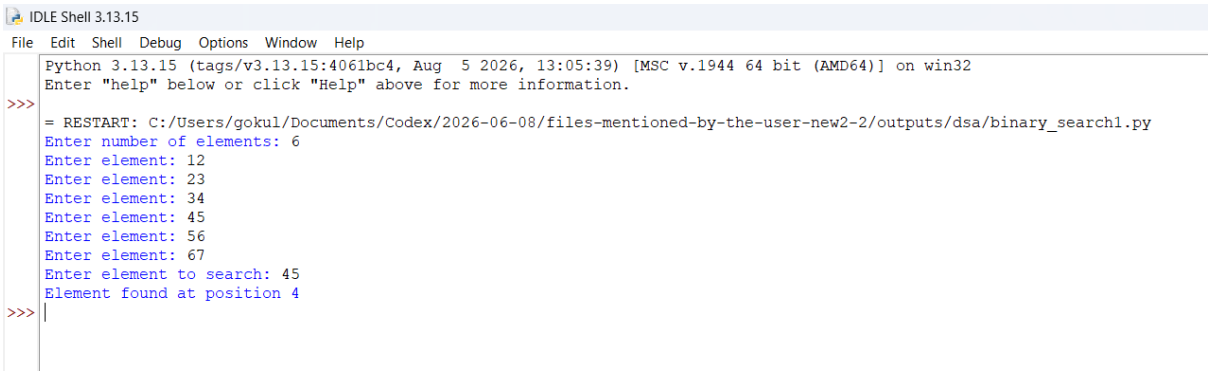
    if arr[mid] == target:
        print("Element found at position", mid + 1)
        found = True
        break
```

```
    elif target < arr[mid]:
        high = mid - 1

    else:
        low = mid + 1

if not found:
    print("Element not found")
```

Output:



Explanation:

- **Best Case:** O(1)
- **Worst Case:** O(log n)
- **Time Complexity:** O(log n)
- **Space Complexity:** O(1)

Reason: In the best case, the target element is the middle element of the array, so it is found in the first comparison. In the worst case, the search range is repeatedly divided into half until the element is found or the range becomes empty. Therefore, the worst-case time complexity is O(log n). Since only a few variables such as low, high, and mid are used, the space complexity is O(1).

Error Table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

c) Write a program to implement Binary Search with and without sorted array as an input.

Program:

```
# Binary Search on an unsorted array

# Taking the number of elements
n = int(input("Enter number of elements: "))

# Taking array elements
arr = []

for i in range(n):
    value = int(input("Enter element: "))
    arr.append(value)

# Sorting the array
arr.sort()

print("Sorted array:", arr)

# Taking the element to search
target = int(input("Enter element to search: "))
```

```
# Binary search
low = 0
high = n - 1
found = False

while low <= high:
    mid = (low + high) // 2

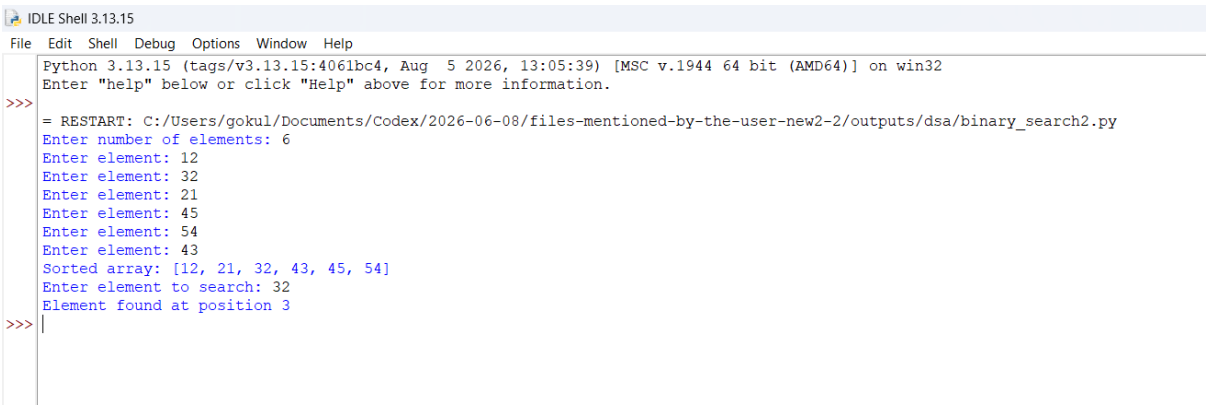
    if arr[mid] == target:
        print("Element found at position", mid + 1)
        found = True
        break

    elif target < arr[mid]:
        high = mid - 1

    else:
        low = mid + 1

if not found:
    print("Element not found")
```

Output:



Explanation:

- **Best Case:** O(1) for the binary search after sorting
- **Worst Case:** O(n log n)
- **Time Complexity:** O(n log n)
- **Space Complexity:** O(n)

Reason: Sorting the array takes O(n log n) time in the average and worst case, while binary search takes O(log n) time. Since sorting takes more time than searching, the overall time complexity is O(n log n). The array requires O(n) space for storing the input elements.

Week-3:

a)write a program to implement Bubble Sort.

Program:

```
# Bubble Sort

# Taking number of elements
n = int(input("Enter number of elements: "))

# Taking array elements
arr = []

for i in range(n):
    value = int(input("Enter element: "))
    arr.append(value)

# Bubble Sort
for i in range(n - 1):
    swapped = False

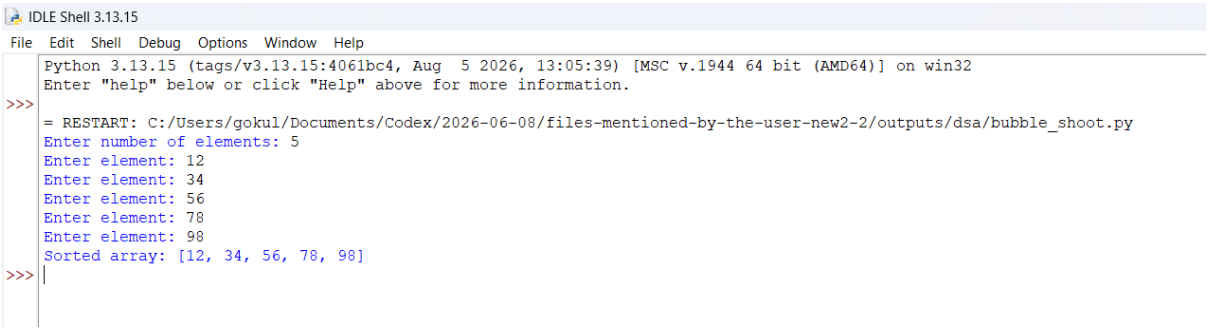
    for j in range(n - 1 - i):
        if arr[j] > arr[j + 1]:
            # Swap elements
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
            swapped = True

    # Stop if array is already sorted
    if not swapped:
```

```
        break

print("Sorted array:", arr)
```

Output:



Explanation:

Complexity

- **Best Case:** O(n)
- **Worst Case:** $O(n^2)$
- **Time Complexity:** $O(n^2)$
- **Space Complexity:** O(1)

Reason: In the best case, when the array is already sorted, the algorithm makes one pass and stops because no swapping occurs, giving O(n) time. In the worst case, when the array is in reverse order, multiple comparisons and swaps are required, giving $O(n^2)$ time. Bubble Sort sorts the elements in the same array and uses only a few extra variables, so its space complexity is O(1).

s.no	Error obtained	Rectification
1.	Break statement written out side the loop	Rectified by writing it in side the loop
2.	Errors in defining the conditions	Rectified using proper

b)Write a program to implement Insertion Sort.

Program:

```
# Insertion Sort

# Taking number of elements
n = int(input("Enter number of elements: "))

# Taking array elements
arr = []

for i in range(n):
    value = int(input("Enter element: "))
    arr.append(value)

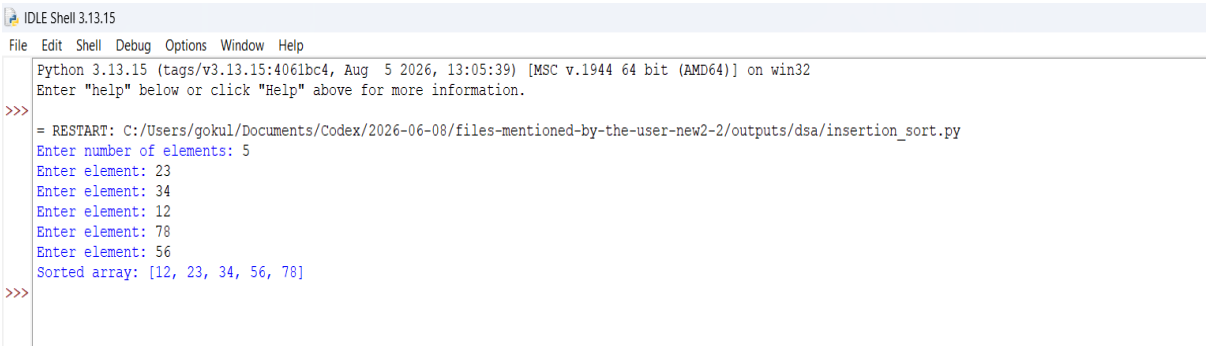
# Insertion Sort
for i in range(1, n):
    key = arr[i]
    j = i - 1

    # Move elements greater than key one position ahead
    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j = j - 1

    # Insert key at correct position
    arr[j + 1] = key
```

```
print("Sorted array:", arr)
```

Output:



Explanation:

- **Best Case:** O(n)
- **Worst Case:** O(n^2)
- **Time Complexity:** O(n^2)
- **Space Complexity:** O(1)

Reason: The best case occurs when the array is already sorted. In this case, each element only needs one comparison, resulting in O(n) time. The worst case occurs when the array is in reverse order, because every element has to be compared with and shifted past all previous elements, resulting in O(n^2) time. The algorithm sorts the array in place and uses only a few extra variables, so the space complexity is O(1).

d) Write a program to implement Selection Sort.

Program:

```
# Selection Sort

# Taking number of elements
n = int(input("Enter number of elements: "))

# Taking array elements
arr = []

for i in range(n):
    value = int(input("Enter element: "))
    arr.append(value)

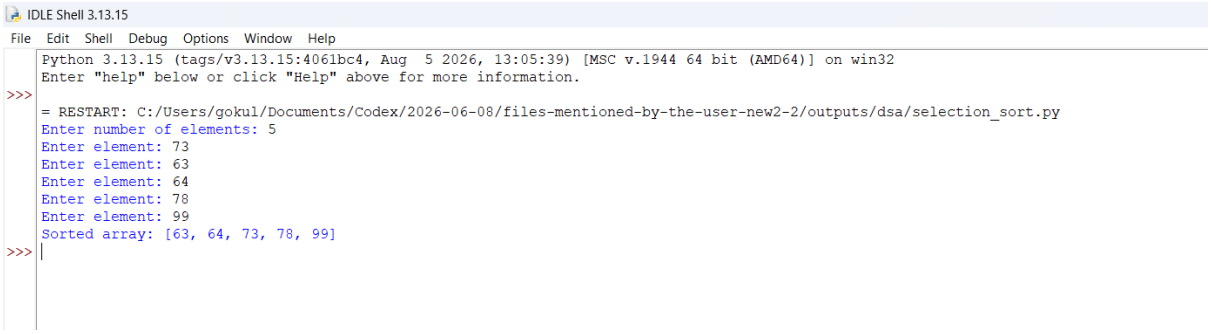
# Selection Sort
for i in range(n - 1):
    min_index = i

    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j

    # Swap minimum element with current element
    arr[i], arr[min_index] = arr[min_index], arr[i]

print("Sorted array:", arr)
```

Output:



Explanation:

- **Best Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Time Complexity:** $O(n^2)$
- **Space Complexity:** $O(1)$

Reason: Selection Sort always searches through the remaining unsorted elements to find the minimum element, even if the array is already sorted. Therefore, the number of comparisons remains almost the same for both best and worst cases, resulting in $O(n^2)$ time complexity. The algorithm performs sorting within the original array and uses only a few extra variables, so its space complexity is $O(1)$.

s.no	Error obtained	Rectification
1.	Break statement written out side the loop	Rectified by writing it in side the loop
2.	Errors in defining the conditions	Rectified using proper

Week-4;

A) write a python program to implement Quick sort, where the elements are taken from User

Aim: write a python program to implement Quick sort, where the elements are taken from user.

Program:

```
def quick_sort(arr):
    # Base case: if the array has 0 or 1 element, it is already sorted
    if len(arr) <= 1:
        return arr

    # Choose the middle element as the pivot
    pivot = arr[len(arr) // 2]

    # Create three lists for partitioning the elements
    left = [x for x in arr if x < pivot]     # Elements smaller than pivot
    middle = [x for x in arr if x == pivot]  # Elements equal to pivot
    right = [x for x in arr if x > pivot]    # Elements greater than pivot

    # Recursively sort the left and right parts, then combine everything
    return quick_sort(left) + middle + quick_sort(right)


# --- Main Program Execution ---
if __name__ == "__main__":
    print("--- Quick Sort Program ---")

    # 1. Ask the user for inputs separated by spaces
```

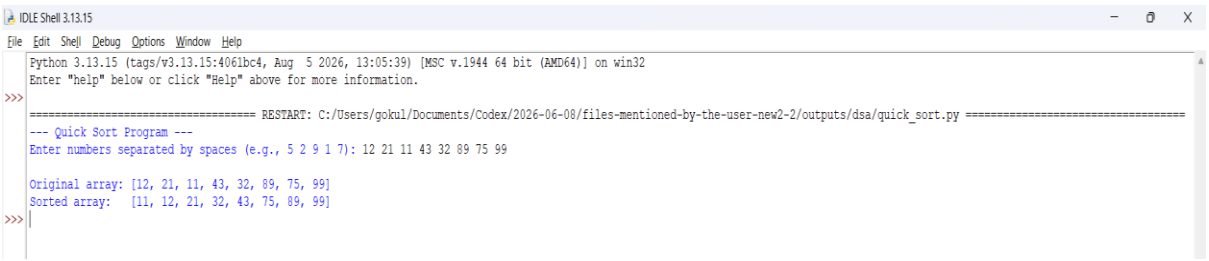
```
user_input = input("Enter numbers separated by spaces (e.g., 5 2 9 1 7): ")


# 2. Convert the input string into a list of integers
# .split() breaks the text by spaces, and int(x) converts each piece to a number
try:
    numbers = [int(x) for x in user_input.split()]


    # Check if the user actually entered any numbers
    if not numbers:
        print("You didn't enter any numbers!")
    else:
        # 3. Sort the numbers and print the final result
        sorted_numbers = quick_sort(numbers)
        print("\nOriginal array:", numbers)
        print("Sorted array:  ", sorted_numbers)


except ValueError:
    print("\n[Error]: Please enter only valid integers separated by spaces.")
```

output:



Explanation:

This Python program uses the Quick Sort algorithm to sort elements in ascending order. It takes n elements from the user and stores them in a list. The first element is chosen as the pivot, and the remaining elements are divided into left (smaller or equal) and right (greater). The function recursively sorts both parts and combines them as left + pivot + right to produce the sorted array.

Error Table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

4B) write a python program to implement merge sort, where the elements are taken from user.

AIM: write a python program to implement merge sort, where the elements are taken from user.

Program:

```
def main():
    print("rollno : cse25126\name : M.Gokul krishna\n")
    n = int(input("Enter size of array: "))
    arr = []
    for i in range(n):
        arr.append(int(input(f"enter {i+1} element: ")))
    quick_sort(arr, 0, n-1)
    print(arr)



def quick_sort(a, low, high):
    if low < high:
        pivot = a[low]      # store pivot VALUE, not index
        i = low - 1
        j = high + 1

        while True:
            i += 1
            while a[i] < pivot:
                i += 1
            j -= 1
            while a[j] > pivot:
                j -= 1
            if i >= j:
                break
```

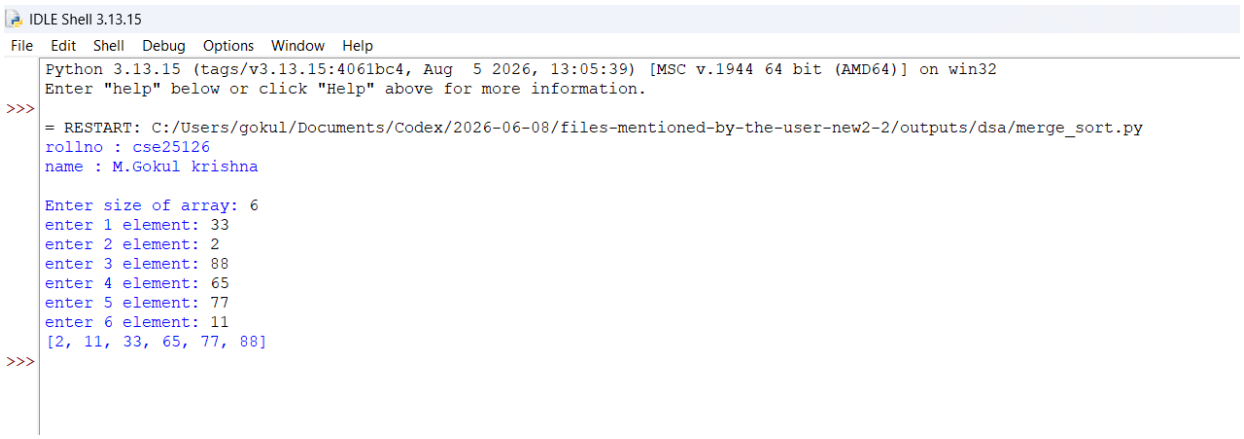
```
            a[i], a[j] = a[j], a[i]


        quick_sort(a, low, j)      # left part (includes j)
        quick_sort(a, j + 1, high) # right part



if __name__ == '__main__':
    main()
```

output:



Explanation:

This Python program uses the Merge Sort algorithm to sort elements in ascending order. It first takes n elements from the user and stores them in a list. The merge_sort() function divides the list into two halves recursively until each sublist has one element. Then, it merges the sublists by comparing elements and placing the smaller one first. The process continues until the entire list is sorted.

Error Table:

s.no	Error obtained	Rectification
1.	Mistakes in induntation	Rectified using proper spaces
2.	Errors in conditions	Rectified

Week 5

5a) Write a program to implement SLL with the below operations. Create linked list, Insert Beginning, Insert End, Insert at Index, delete by value, delete first node, delete last node, count number of nodes, display/transverse exit.

AIM: Write a program to implement SLL with the below operations. Create linked list, Insert Beginning, Insert End, Insert at Index, delete by value, delete first node, delete last node, count number of nodes, display/transverse exit.

Program:

```
def main():
    print("CSE25126 M.Gokul krishna\n")
    my_LinkedList = None
    while True:
        print(
            "1.create\n2.insert at beginning\n3.insert at end\n4.insert at certain position\n5.delete first node\n6.delete last node\n7.delete specific node\n"
            "8.count\n9.display\n10.exit"
        )
        option = input("Enter your choice: ")
        if option == "1":
            my_LinkedList = LinkedList()
            print("linked list successfully created")
        elif my_LinkedList is None:
            print("please create the linked list first (option 1)")
        elif option == "2":
            my_LinkedList.insert_begin(int(input("Enter the data: ")))
            print("element is inserted")
        elif option == "3":
            my_LinkedList.insert_end(int(input("Enter the data: ")))
            print("element inserted")
        elif option == "4":
```

```
            my_LinkedList.insert_position(
                int(input("Enter the data: ")), int(input("Enter the position: "))
            )
        elif option == "5":
            my_LinkedList.delete_begin()
        elif option == "6":
            my_LinkedList.delete_end()
        elif option == "7":
            my_LinkedList.delete_position(int(input("Enter position: ")))
        elif option == "8":
            count = my_LinkedList.size()
            print("the size of linked list is {}".format(count))
        elif option == "9":
            my_LinkedList.display()
        elif option == "10":
            print("exiting the program")
            break
        else:
            print("option not defined")


class LinkedList:
    def __init__(self):
        self.head = None

    def insert_begin(self, data):
        node = Node(data)
        if self.head is None:
            self.head = node
```

```
        return
    node.next = self.head
    self.head = node

def insert_end(self, data):
    if self.head is None:
        self.insert_begin(data)
        return
    node = Node(data)
    temp = self.head
    while temp.next:
        temp = temp.next
    temp.next = node

def size(self):
    temp = self.head
    count = 0
    while temp:
        temp = temp.next
        count += 1
    return count

def insert_position(self, data, pos):
    # positions are 1-indexed: pos=1 means new head
    n = self.size()
    if pos < 1 or pos > n + 1:
        print("position not available")
        return
    if pos == 1:
```

```
            self.insert_begin(data)
            print("element inserted")
            return
        node = Node(data)
        temp = self.head
        count = 1
        while count < pos - 1:
            temp = temp.next
            count += 1
        node.next = temp.next
        temp.next = node
        print("element inserted")

    def display(self):
        if self.head is None:
            print("The linked list is empty")
            return
        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")

    def delete_begin(self):
        if self.head is None:
            print("The Linked list is empty")
            return
        self.head = self.head.next
        print("first node deleted")
```

```
def delete_end(self):
    if self.head is None:
        print("The Linked list is empty")
        return
    if self.head.next is None:
        self.head = None
        print("last node deleted")
        return
    temp = self.head
    while temp.next.next:
        temp = temp.next
    temp.next = None
    print("last node deleted")

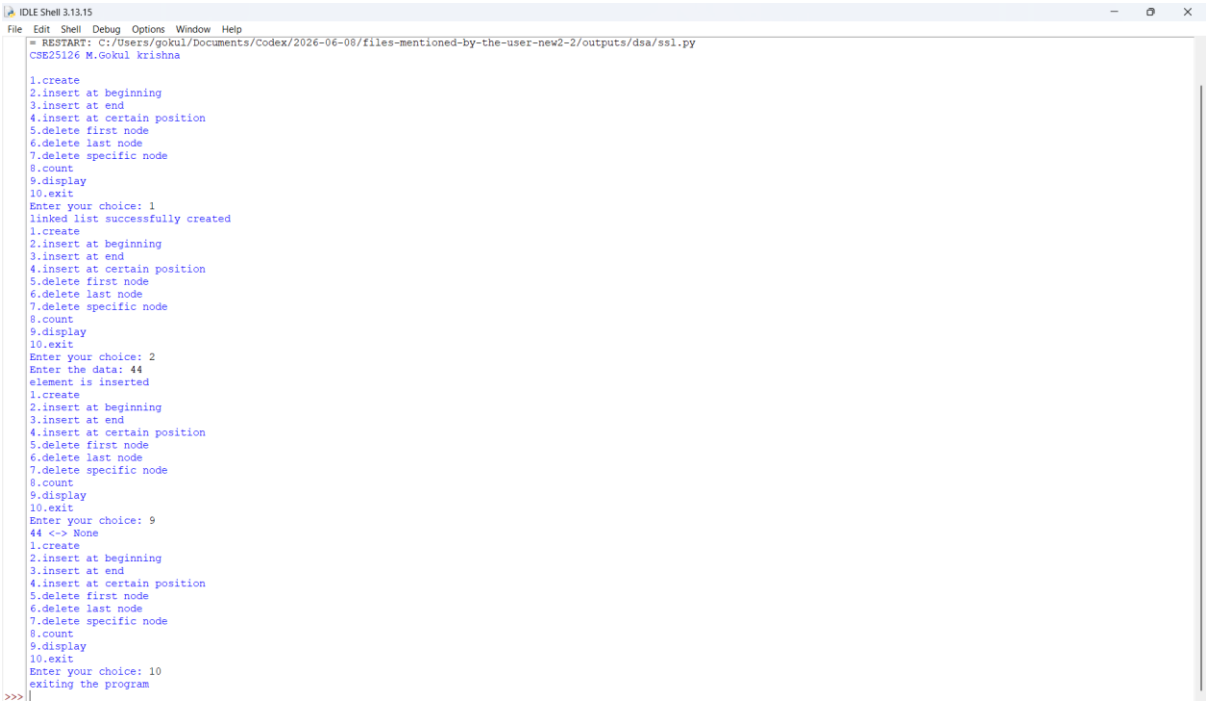
def delete_position(self, pos):
    # positions are 1-indexed: pos=1 means head
    if self.head is None:
        print("Empty linked list")
        return
    n = self.size()
    if pos < 1 or pos > n:
        print("position does not exist")
        return
    if pos == 1:
        self.head = self.head.next
        print("node at position is deleted")
        return
    temp = self.head
```

```
        count = 1
        while count < pos - 1:
            temp = temp.next
            count += 1
        temp.next = temp.next.next
        print("node at position is deleted")


class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


if __name__ == "__main__":
    main()
```

output:



Explanation:

This Python program implements a Singly Linked List (SLL) using a menu-driven approach. Each node contains data and a next pointer, while the head points to the first node. The program supports creating the list, inserting nodes (beginning, end, index), deleting nodes, counting nodes, and displaying the list. The display output is in the format:

Error Table:

s.no	Error obtained	Rectification
2.	Errors in conditions	Rectified by taking proper
3.	Errors in node linking	Rectified
4.	Errors in insertion and deletion	Rectified
5.	Errors in index handling	Rectified

Week 6

Q)Write a python program to implement stack operations such as push, pop, peek, display using an array and a linked list

AIM: a python program to implement stack operations such as push, pop, peek, display using an array and a linked list

Stack using arrays:

Program:

```
class StackArray:
  def __init__(self):
    self.stack = []

  def push(self, data):
    self.stack.append(data)
    print(data, "pushed")

  def pop(self):
    if len(self.stack) == 0:
      print("Stack is empty")
    else:
      print("Popped:", self.stack.pop())

  def peek(self):
    if len(self.stack) == 0:
      print("Stack is empty")
    else:
      print("Top element:", self.stack[-1])

  def display(self):
```

```
        if len(self.stack) == 0:
            print("Stack is empty")
        else:
            print("Stack:", end=" ")
            for i in range(len(self.stack) - 1, -1, -1):
                print(self.stack[i], end=" ")
            print()


s = StackArray()

while True:
    print("\n--- STACK USING ARRAY ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        s.push(value)

    elif choice == 2:
        s.pop()
```

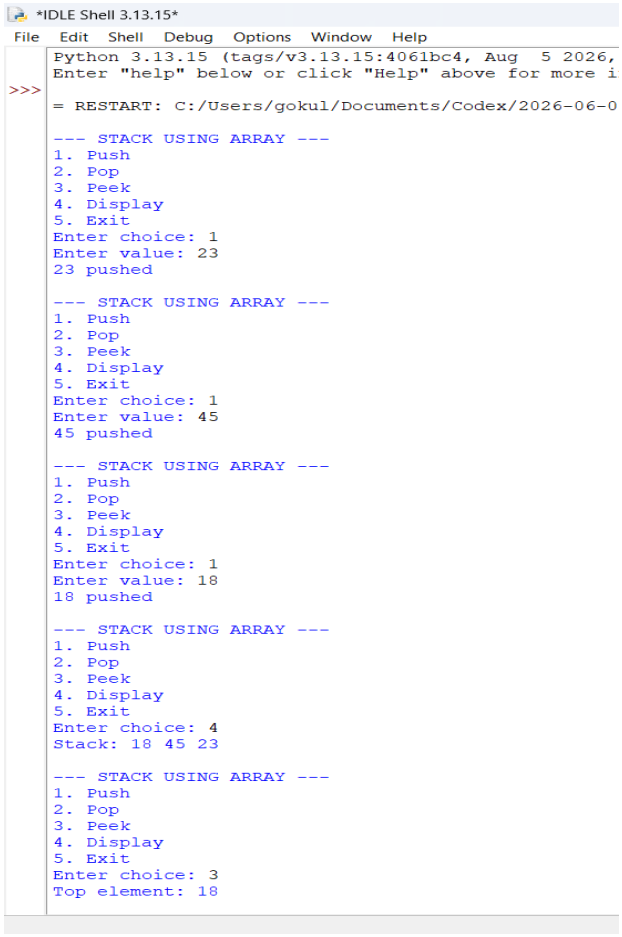
```
elif choice == 3:
    s.peek()

elif choice == 4:
    s.display()

elif choice == 5:
    print("Program Ended")
    break

else:
    print("Invalid Choice")
```

Output:


IDLE Shell 3.13.15
File Edit Shell Debug Options Window Help
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug 5 2026,
Enter "help" below or click "Help" above for more i
>>>
= RESTART: C:/Users/gokul/Documents/Codex/2026-06-0
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 23
23 pushed
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 45
45 pushed
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 18
18 pushed
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 4
Stack: 18 45 23
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 3
Top element: 18

Stacks using linked lists:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class StackLinkedList:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(data, "pushed")

    def pop(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Popped:", self.top.data)
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)
```

```
    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack:", end=" ")
            while temp:
                print(temp.data, end=" ")
                temp = temp.next
            print()


s = StackLinkedList()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        s.push(value)
```

```
elif choice == 2:
    s.pop()

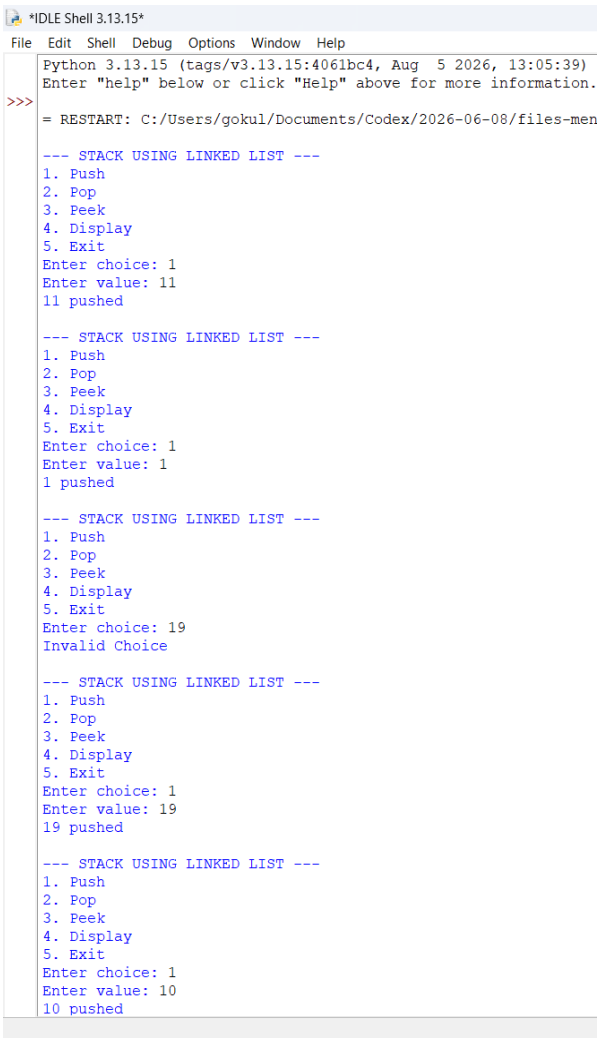
elif choice == 3:
    s.peek()

elif choice == 4:
    s.display()

elif choice == 5:
    print("Program Ended")
    break

else:
    print("Invalid Choice")
```

Out Put:



```
*IDLE Shell 3.13.15*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug  5 2026, 13:05:39)
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/gokul/Documents/Codex/2026-06-08/files-men

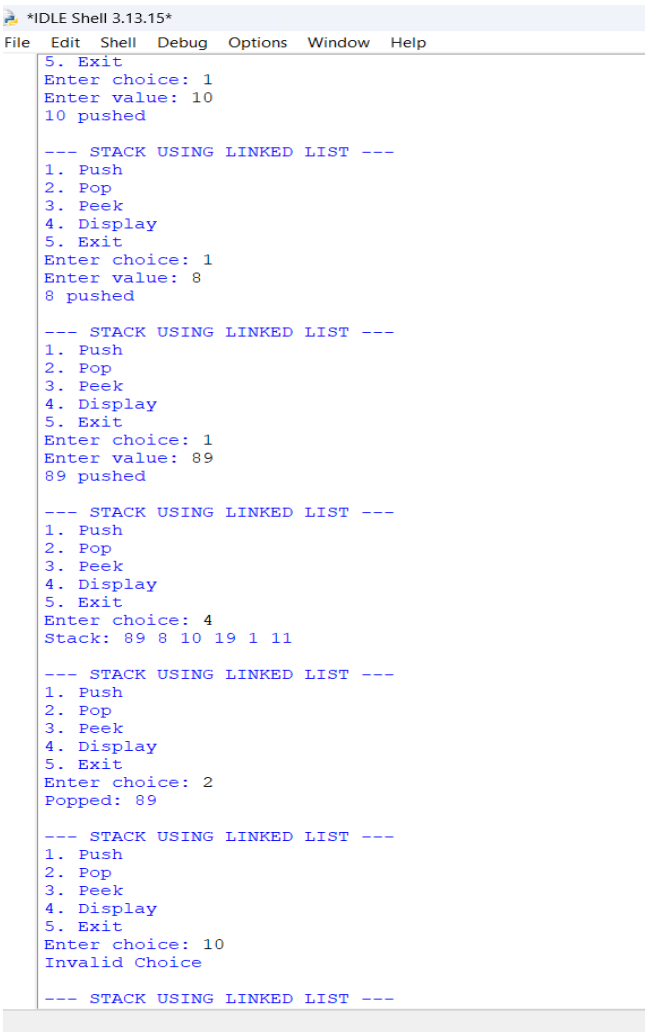
--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 11
11 pushed

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 1
1 pushed

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 19
Invalid Choice

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 19
19 pushed

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 10
10 pushed
```



```
*IDLE Shell 3.13.15*
File  Edit  Shell  Debug  Options  Window  Help
5. Exit
Enter choice: 1
Enter value: 10
10 pushed

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 8
8 pushed

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value: 89
89 pushed

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 4
Stack: 89 8 10 19 1 11

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 2
Popped: 89

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 10
Invalid Choice

--- STACK USING LINKED LIST ---
```

Explanation:

program implements a Stack using a list. It first creates an empty list called stack and sets the maximum size using the MAX variable. A while loop displays the menu repeatedly until the user chooses Exit.

Inside the loop, the user's choice is taken using input(). The program uses if-elif statements to perform different operations. In Push, it checks whether the stack is full; if not, it inserts the element using stack.append(). In Pop, it checks whether the stack is empty and removes the top element using stack.pop(). The Peek operation displays the top element using stack[-1] without removing it, and Display prints all elements from the top to the bottom of the stack.

The program also handles overflow and underflow conditions prevent invalid operations.

S.No	Error Obtained	Rectification
1	Errors in conditions	Fixed the conditional statements.
2	Errors in Push and Pop	Used correct push and pop operations.
3	Errors in top pointer	Corrected top element handling.
4	Errors in Display	Fixed the display logic.

Week-7

A)write a python program to implement a queue and its operations such as enque,dequeue , peek and display with a menu driven approach implement using array and linked list data structure

AIM: To write a python program to implement a queue and its operations such as enque,dequeue , peek and display with a menu driven approach implement using array and linked list data structure

Program:

USING ARRAYS:

```
class ArrayQueue:
  def __init__(self, size):
    self.size = size
    self.queue = [None] * size
    self.front = -1
    self.rear = -1

  def enqueue(self, item):
    if self.rear == self.size - 1:
      print("Queue Overflow")
    else:
      if self.front == -1:
        self.front = 0
      self.rear += 1
      self.queue[self.rear] = item
      print(item, "inserted")
```

```
def dequeue(self):
    if self.front == -1 or self.front > self.rear:
        print("Queue Underflow")
    else:
        item = self.queue[self.front]
        self.front += 1

        if self.front > self.rear:
            self.front = self.rear = -1

        print(item, "deleted")

def peek(self):
    if self.front == -1:
        print("Queue is Empty")
    else:
        print("Front element =", self.queue[self.front])

def display(self):
    if self.front == -1:
        print("Queue is Empty")
    else:
        print("Queue:", end=" ")
        for i in range(self.front, self.rear + 1):
            print(self.queue[i], end=" ")
        print()
```

```
# -------- Main Program --------
size = int(input("Enter Queue Size: "))
q = ArrayQueue(size)

while True:
    print("\n--- ARRAY QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

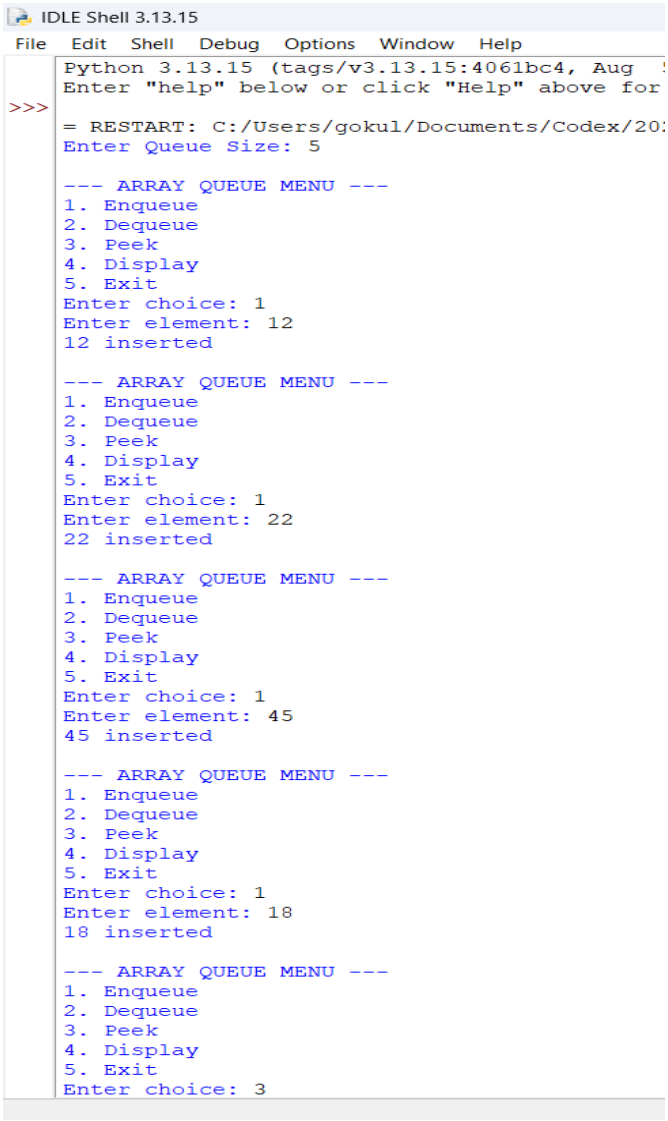
    if choice == 1:
        item = int(input("Enter element: "))
        q.enqueue(item)

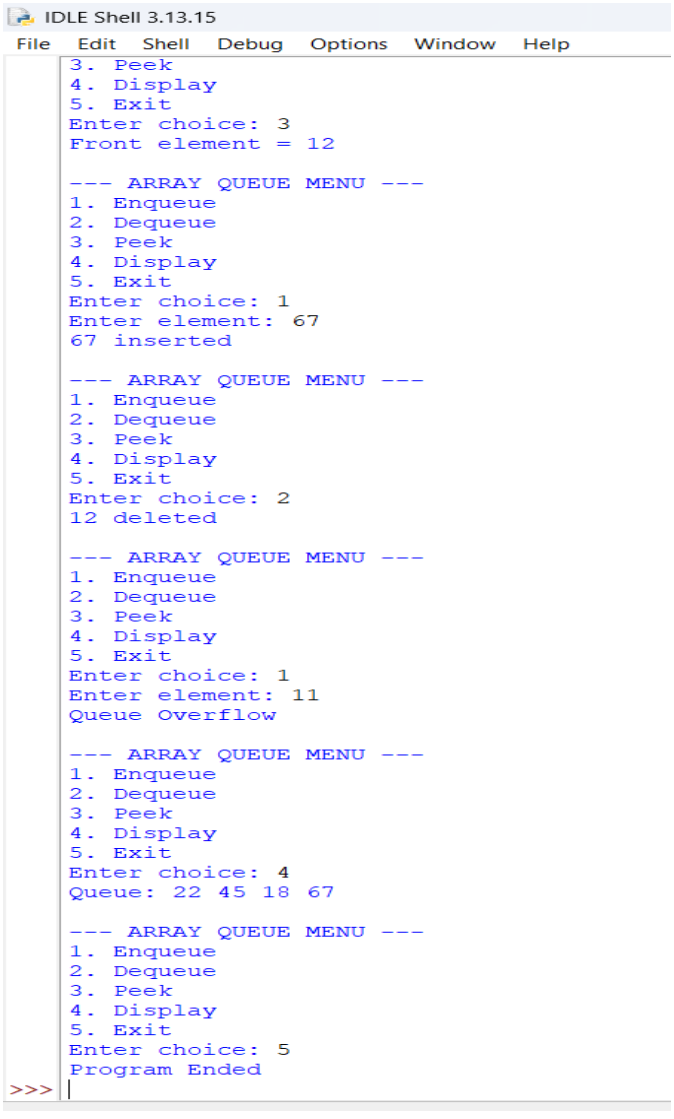
    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()
```

```
    elif choice == 4:
        q.display()
    elif choice == 5:
        print("Program Ended")
        break
    else:
        print("Invalid Choice")
```

Output:





USING LINKED LISTS:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedQueue:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, item):
        new_node = Node(item)

        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print(item, "inserted")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
            return
```

```
        item = self.front.data
        self.front = self.front.next

        if self.front is None:
            self.rear = None

        print(item, "deleted")

    def peek(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            print("Front element =", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is Empty")
            return

        temp = self.front
        print("Queue:", end=" ")
        while temp:
            print(temp.data, end=" ")
            temp = temp.next
        print()


# -------- Main Program --------
q = LinkedQueue()
```

```
while True:
    print("\n--- LINKED LIST QUEUE MENU ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        item = int(input("Enter element: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

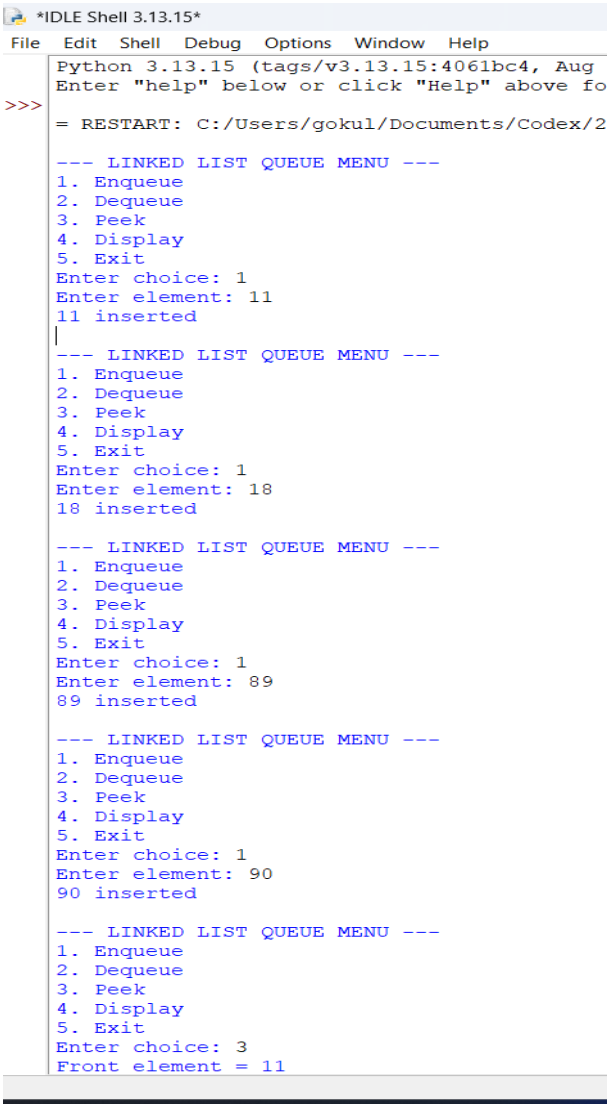
    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Program Ended")
        break

    else:
```

```
print("Invalid Choice")
```

Output:



```
*IDLE Shell 3.13.15*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.15 (tags/v3.13.15:4061bc4, Aug
Enter "help" below or click "Help" above fo
>>>
= RESTART: C:/Users/gokul/Documents/Codex/2

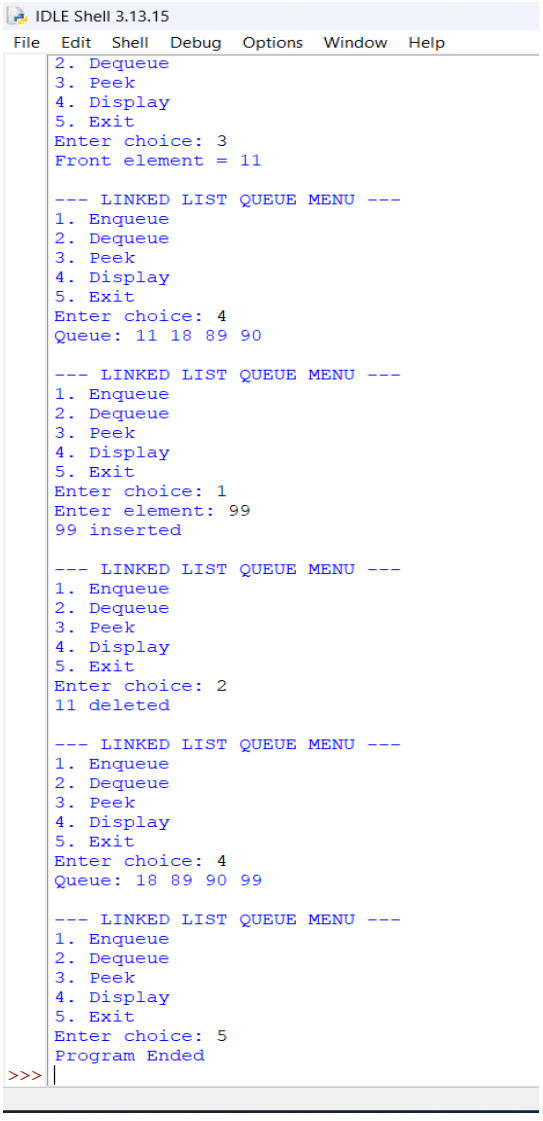
--- LINKED LIST QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter element: 11
11 inserted

--- LINKED LIST QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter element: 18
18 inserted

--- LINKED LIST QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter element: 89
89 inserted

--- LINKED LIST QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter element: 90
90 inserted

--- LINKED LIST QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 3
Front element = 11
```



```
IDLE Shell 3.13.15
File  Edit  Shell  Debug  Options  Window  Help
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 3
Front element = 11

--- LINKED LIST QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 4
Queue: 11 18 89 90

--- LINKED LIST QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter element: 99
99 inserted

--- LINKED LIST QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 2
11 deleted

--- LINKED LIST QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 4
Queue: 18 89 90 99

--- LINKED LIST QUEUE MENU ---
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 5
Program Ended
>>>
```

Explanation:

This program implements a Queue ADT using both Array and Singly Linked List with a menu-driven approach. It performs the basic queue operations: Enqueue (insert), Dequeue (delete), Peek (view the front element), and Display (traverse the queue). The queue follows the FIFO (First In, First Out) principle, where elements are inserted at the rear and removed from the front.

ErrorTable:

S.no	Error Obtained	rectificantion
1	Queue Overflow while inserting	Added overflow condition before enqueue()
2	Queue Underflow while deleting	Checked whether the queue is empty before dequeue()
3	front and rear not reset after deletion	Reset both pointers to -1 (or None in linked list) when the queue became empty

B)write a python program to implement a circular queue and its operations such as enqueue, dequeue , peek and display with a menu driven approach implement using array and linked list data structure

AIM:

To write a python program to implement a circular queue and its operations such as enqueue, dequeue , peek and display with a menu driven approach implement using array and linked list data structure

Program:

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
```

```
            print("Queue Overflow")
            return

        if self.front == -1:
            self.front = self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.size

        self.queue[self.rear] = item
        print(item, "inserted")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
            return

        item = self.queue[self.front]

        if self.front == self.rear:
            self.front = self.rear = -1
        else:
            self.front = (self.front + 1) % self.size

        print(item, "deleted")

    def peek(self):
        if self.front == -1:
            print("Queue is Empty")
        else:
```

```
            print("Front element =", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is Empty")
            return

        print("Circular Queue:", end=" ")
        i = self.front
        while True:
            print(self.queue[i], end=" ")
            if i == self.rear:
                break
            i = (i + 1) % self.size
        print()


# -------- MAIN --------
size = int(input("Enter Queue Size: "))
q = CircularQueue(size)

while True:
    print("\n--- CIRCULAR QUEUE (ARRAY) ---")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```

```
ch = int(input("Enter choice: "))

if ch == 1:
    x = int(input("Enter element: "))
    q.enqueue(x)

elif ch == 2:
    q.dequeue()

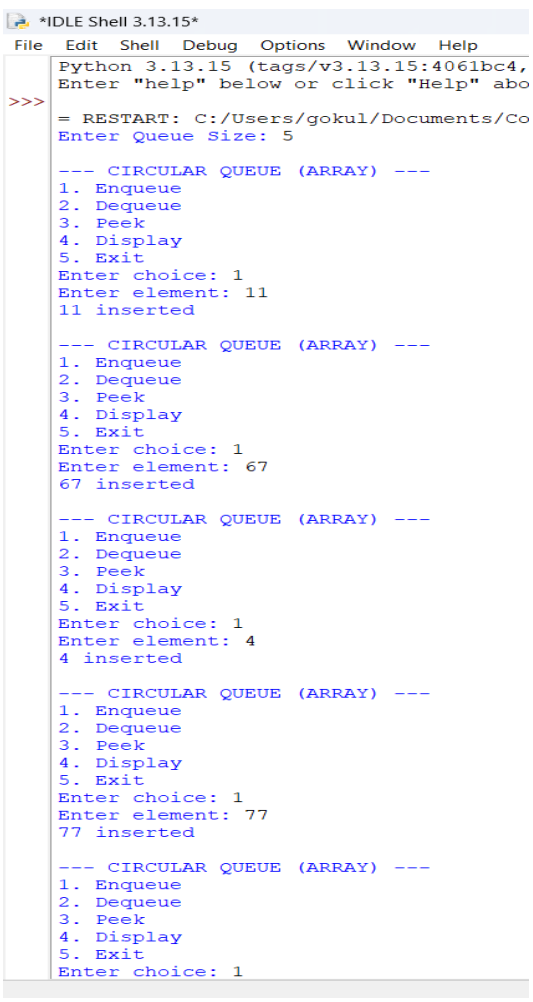
elif ch == 3:
    q.peek()

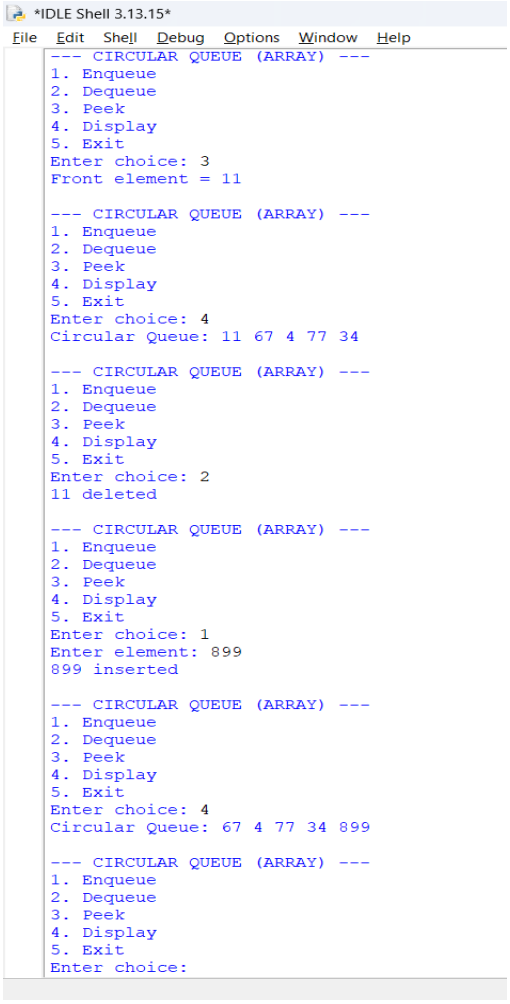
elif ch == 4:
    q.display()

elif ch == 5:
    break

else:
    print("Invalid Choice")
```

OutPut:





USING LINKED LISTS:

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedQueue:
    def __init__(self):
        self.front = None
        self.rear = None
```

```
def enqueue(self, item):
    new = Node(item)

    if self.front is None:
        self.front = self.rear = new
        self.rear.next = self.front
    else:
        self.rear.next = new
        self.rear = new
        self.rear.next = self.front

    print(item, "inserted")

def dequeue(self):
    if self.front is None:
        print("Queue Underflow")
        return

    item = self.front.data

    if self.front == self.rear:
        self.front = self.rear = None
    else:
        self.front = self.front.next
        self.rear.next = self.front

    print(item, "deleted")
```

```
    def peek(self):
        if self.front is None:
            print("Queue is Empty")
        else:
            print("Front element =", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is Empty")
            return

        temp = self.front
        print("Circular Queue:", end=" ")

        while True:
            print(temp.data, end=" ")
            temp = temp.next
            if temp == self.front:
                break
        print()


# -------- MAIN --------
q = CircularLinkedQueue()

while True:
    print("\n--- CIRCULAR QUEUE (LINKED LIST) ---")
    print("1. Enqueue")
    print("2. Dequeue")
```

```
print("3. Peek")
print("4. Display")
print("5. Exit")

ch = int(input("Enter choice: "))

if ch == 1:
    x = int(input("Enter element: "))
    q.enqueue(x)

elif ch == 2:
    q.dequeue()

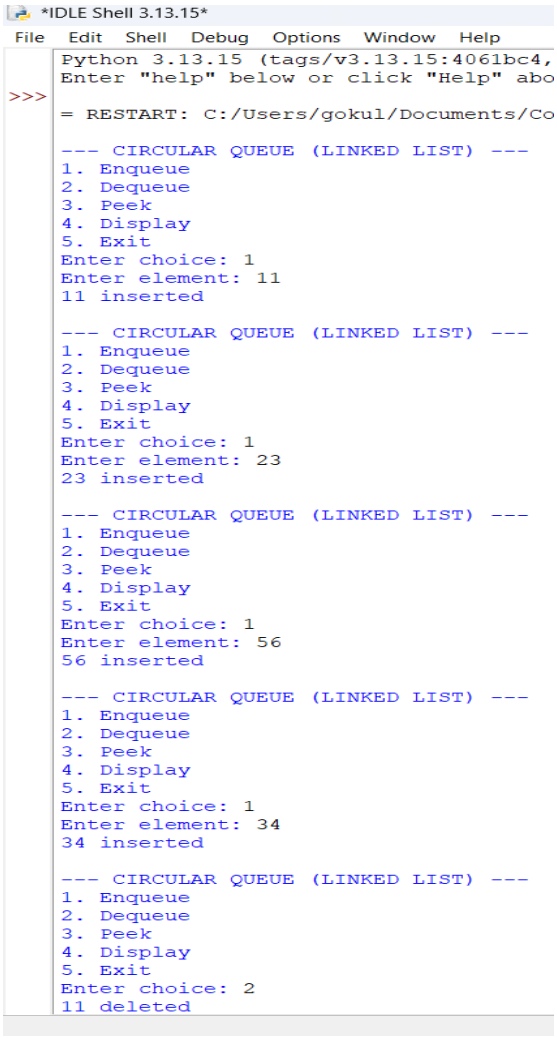
elif ch == 3:
    q.peek()

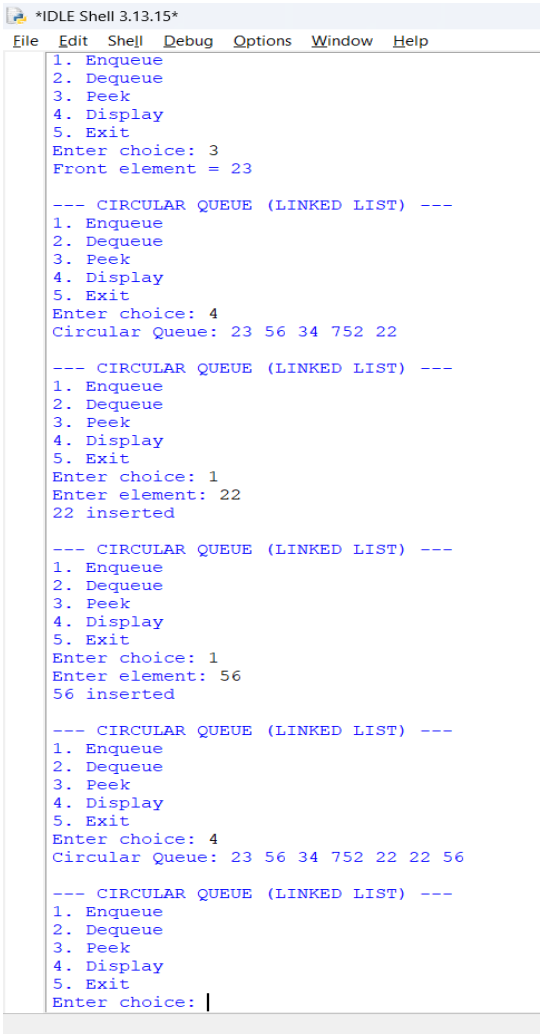
elif ch == 4:
    q.display()

elif ch == 5:
    break

else:
    print("Invalid Choice")
```

OutPut:





Explanation:

This program implements a Circular Queue using both Array and Circular Singly Linked List with a menu-driven approach. It performs the operations Enqueue, Dequeue, Peek, and Display while following the FIFO (First In First Out) principle. In the array implementation, the front and rear indices move circularly using the modulo operator, whereas in the linked list implementation the rear node is linked back to the front node to form a circular structure.

Error Table:

S.no	Error Obtained	Rectification
1	Queue Overflow during enqueue	Checked the full condition before insertion.
2	Queue Underflow during dequeue	Verified that the queue is not empty before deletion
3	Display entered an infinite loop	Stopped traversal when the rear/front node was reached again

Week-8

Q) write a python program to implement a binary tree with its operations using a array and a linked list

AIM: to write a python program to implement a binary tree with its operations using a array and a linked list

Arrays:

Code:

```
class BinaryTree:
  def __init__(self, size):
    self.tree = [None] * size
    self.size = size

  # Insert a node
  def insert(self, value):
    for i in range(self.size):
      if self.tree[i] is None:
        self.tree[i] = value
        return
    print("Tree is full")

  # Search a node
  def search(self, value):
    for i in range(self.size):
      if self.tree[i] == value:
        return i
    return -1
```

```
# Inorder Traversal
def inorder(self, i=0):
    if i >= self.size or self.tree[i] is None:
        return

    self.inorder(2 * i + 1)
    print(self.tree[i], end=" ")
    self.inorder(2 * i + 2)

# Preorder Traversal
def preorder(self, i=0):
    if i >= self.size or self.tree[i] is None:
        return

    print(self.tree[i], end=" ")
    self.preorder(2 * i + 1)
    self.preorder(2 * i + 2)

# Postorder Traversal
def postorder(self, i=0):
    if i >= self.size or self.tree[i] is None:
        return

    self.postorder(2 * i + 1)
    self.postorder(2 * i + 2)
    print(self.tree[i], end=" ")

# Display array
```

```
    def display(self):
        print(self.tree)


# Main program
bt = BinaryTree(10)

bt.insert(25)
bt.insert(15)
bt.insert(40)
bt.insert(10)
bt.insert(20)
bt.insert(30)
bt.insert(50)

print("Array representation:")
bt.display()

print("\nInorder:")
bt.inorder()

print("\nPreorder:")
bt.preorder()

print("\nPostorder:")
bt.postorder()

print("\n\nSearching for 30:")
```

```
position = bt.search(30)

if position != -1:
    print("30 found at index", position)
else:
    print("30 not found")
```

Output:



Using Linked List:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    # Insert a node
    def insert(self, value):
```

```
    new_node = Node(value)

    if self.root is None:
        self.root = new_node
        return

    queue = [self.root]

    while queue:
        current = queue.pop(0)

        if current.left is None:
            current.left = new_node
            return
        else:
            queue.append(current.left)

        if current.right is None:
            current.right = new_node
            return
        else:
            queue.append(current.right)

# Search a node
def search(self, value):
    if self.root is None:
        return False
```

```
    queue = [self.root]

    while queue:
        current = queue.pop(0)

        if current.data == value:
            return True

        if current.left:
            queue.append(current.left)

        if current.right:
            queue.append(current.right)

    return False

# Inorder Traversal
def inorder(self, node):
    if node is not None:
        self.inorder(node.left)
        print(node.data, end=" ")
        self.inorder(node.right)

# Preorder Traversal
def preorder(self, node):
    if node is not None:
        print(node.data, end=" ")
        self.preorder(node.left)
```

```
        self.preorder(node.right)

    # Postorder Traversal
    def postorder(self, node):
        if node is not None:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

    # Level Order Traversal
    def level_order(self):
        if self.root is None:
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            print(current.data, end=" ")

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)
```

```
# Main program
bt = BinaryTree()

bt.insert(25)
bt.insert(15)
bt.insert(40)
bt.insert(10)
bt.insert(20)
bt.insert(30)
bt.insert(50)

print("Inorder:")
bt.inorder(bt.root)

print("\nPreorder:")
bt.preorder(bt.root)

print("\nPostorder:")
bt.postorder(bt.root)

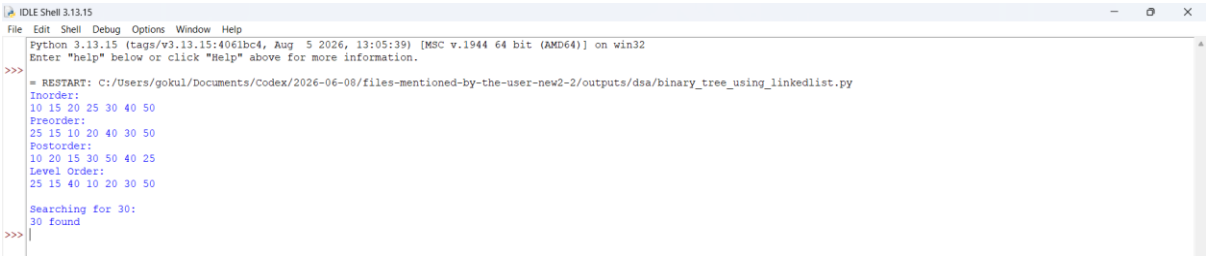
print("\nLevel Order:")
bt.level_order()

print("\n\nSearching for 30:")

if bt.search(30):
   print("30 found")
else:
```

```
print("30 not found")
```

Output:



Explanation:

A **Binary Tree** is a non-linear data structure in which each node can have a maximum of two children, called the **left child** and **right child**. It can be implemented using an **array** or a **linked list**. The main operations include **insertion, searching, and traversal**. The tree can be traversed using **Inorder, Preorder, Postorder, and Level Order** methods. In array representation, nodes are stored using index positions, whereas in linked-list representation, each node contains **data and pointers to its left and right children**. Both methods are useful for storing and processing hierarchical data.

Inorder: Left → Root → Right

Preorder: Root → Left → Right

Postorder: Left → Right → Root

Level Order: Visit nodes level by level

Error Table

S.no	Error	Explanation
1	Using 2*i+1 incorrectly	Left = 2*i+1, Right = 2*i+2
2	Using wrong variable name	Use the same variable name throughout the program.

Week -9

Q) write a python program to implement a binary search treee with its operation and traversals

AIM: to write a python program to implement a binary search treee with its operation and traversals

Code:

```
# Binary Search Tree


class Node:
  def __init__(self, data):
    self.data = data
    self.left = None
    self.right = None



class BinarySearchTree:


  # Insert a node
  def insert(self, root, data):
    if root is None:
      return Node(data)


    if data < root.data:
      root.left = self.insert(root.left, data)
    elif data > root.data:
      root.right = self.insert(root.right, data)


    return root
```

```
# Search for a node
def search(self, root, data):
    if root is None:
        return False

    if root.data == data:
        return True
    elif data < root.data:
        return self.search(root.left, data)
    else:
        return self.search(root.right, data)

# Find minimum value node
def find_min(self, root):
    current = root

    while current.left is not None:
        current = current.left

    return current

# Delete a node
def delete(self, root, data):
    if root is None:
        return root

    if data < root.data:
```

```
            root.left = self.delete(root.left, data)

        elif data > root.data:
            root.right = self.delete(root.right, data)

        else:
            # Case 1: No child
            if root.left is None and root.right is None:
                return None

            # Case 2: Only right child
            elif root.left is None:
                return root.right

            # Case 3: Only left child
            elif root.right is None:
                return root.left

            # Case 4: Two children
            else:
                temp = self.find_min(root.right)
                root.data = temp.data
                root.right = self.delete(root.right, temp.data)

        return root

    # Inorder: Left -> Root -> Right
    def inorder(self, root):
```

```
        if root is not None:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    # Preorder: Root -> Left -> Right
    def preorder(self, root):
        if root is not None:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    # Postorder: Left -> Right -> Root
    def postorder(self, root):
        if root is not None:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")


# Main program
bst = BinarySearchTree()
root = None

while True:
    print("\n\n--- BINARY SEARCH TREE ---")
    print("1. Insert")
    print("2. Search")
```

```
print("3. Delete")
print("4. Inorder Traversal")
print("5. Preorder Traversal")
print("6. Postorder Traversal")
print("7. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    data = int(input("Enter value to insert: "))
    root = bst.insert(root, data)
    print("Node inserted successfully.")

elif choice == 2:
    data = int(input("Enter value to search: "))

    if bst.search(root, data):
        print("Element found.")
    else:
        print("Element not found.")

elif choice == 3:
    data = int(input("Enter value to delete: "))
    root = bst.delete(root, data)
    print("Node deleted successfully.")

elif choice == 4:
    print("Inorder Traversal:")
```

```
        bst.inorder(root)

    elif choice == 5:
        print("Preorder Traversal:")
        bst.preorder(root)

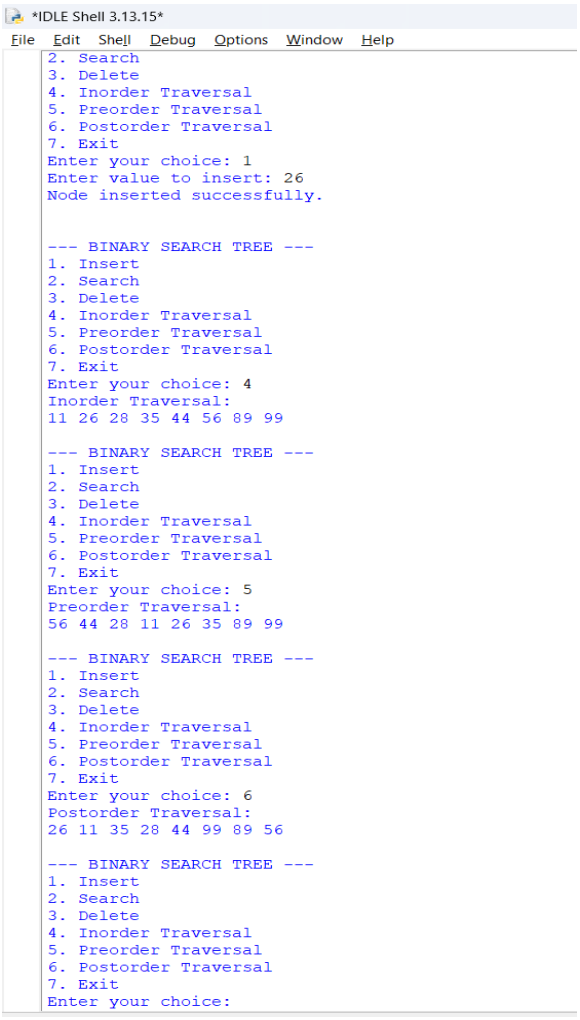
    elif choice == 6:
        print("Postorder Traversal:")
        bst.postorder(root)

    elif choice == 7:
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")
```

Output:





Explanation:

This Python program implements a Binary Search Tree (BST) using nodes and supports insertion, searching, and deletion of elements. It also performs inorder, preorder, and postorder traversals of the tree. In a BST, smaller values are stored in the left subtree and larger values in the right subtree. The program uses a menu-driven approach to perform different operations.

Error Table:

S.no	Error	Explanation
1	Using wrong comparison operator	Use < for the left subtree and > for the right subtree.
2	Forgetting return root	Return the updated root after insertion or deletion.
3	Using wrong traversal order	Inorder = Left → Root → Right; Preorder = Root → Left → Right; Postorder = Left → Right → Root.

Week-10

10)Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing)

Aim: To implement Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).

Code:

```
# WEEK 10 - HASHING

# Separate Chaining, Linear Probing and Quadratic Probing


# Function for Separate Chaining

def separate_chaining():

  size = int(input("Enter hash table size: "))


  table = [[] for _ in range(size)]


  n = int(input("Enter number of elements: "))


  for i in range(n):

    key = int(input("Enter key: "))

    index = key % size

    table[index].append(key)


  print("\nSeparate Chaining:")

  for i in range(size):

    print(i, ":", table[i])



# Function for Linear Probing
```

```
def linear_probing():
    size = int(input("Enter hash table size: "))

    table = [-1] * size

    n = int(input("Enter number of elements: "))

    for i in range(n):
        key = int(input("Enter key: "))

        index = key % size
        start = index

        while table[index] != -1:
            index = (index + 1) % size

            if index == start:
                print("Hash table is full!")
                break

        else:
            table[index] = key

    print("\nLinear Probing:")
    for i in range(size):
        print(i, ":", table[i])
```

```
# Function for Quadratic Probing
def quadratic_probing():
    size = int(input("Enter hash table size: "))

    table = [-1] * size

    n = int(input("Enter number of elements: "))

    for i in range(n):
        key = int(input("Enter key: "))

        hash_index = key % size
        j = 0

        while j < size:
            index = (hash_index + j * j) % size

            if table[index] == -1:
                table[index] = key
                break

            j = j + 1

        if j == size:
            print("Hash table is full!")

    print("\nQuadratic Probing:")
    for i in range(size):
```

```
        print(i, ":", table[i])



# Main program
while True:

    print("\n========== HASHING ==========")
    print("1. Separate Chaining")
    print("2. Linear Probing")
    print("3. Quadratic Probing")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        separate_chaining()

    elif choice == 2:
        linear_probing()

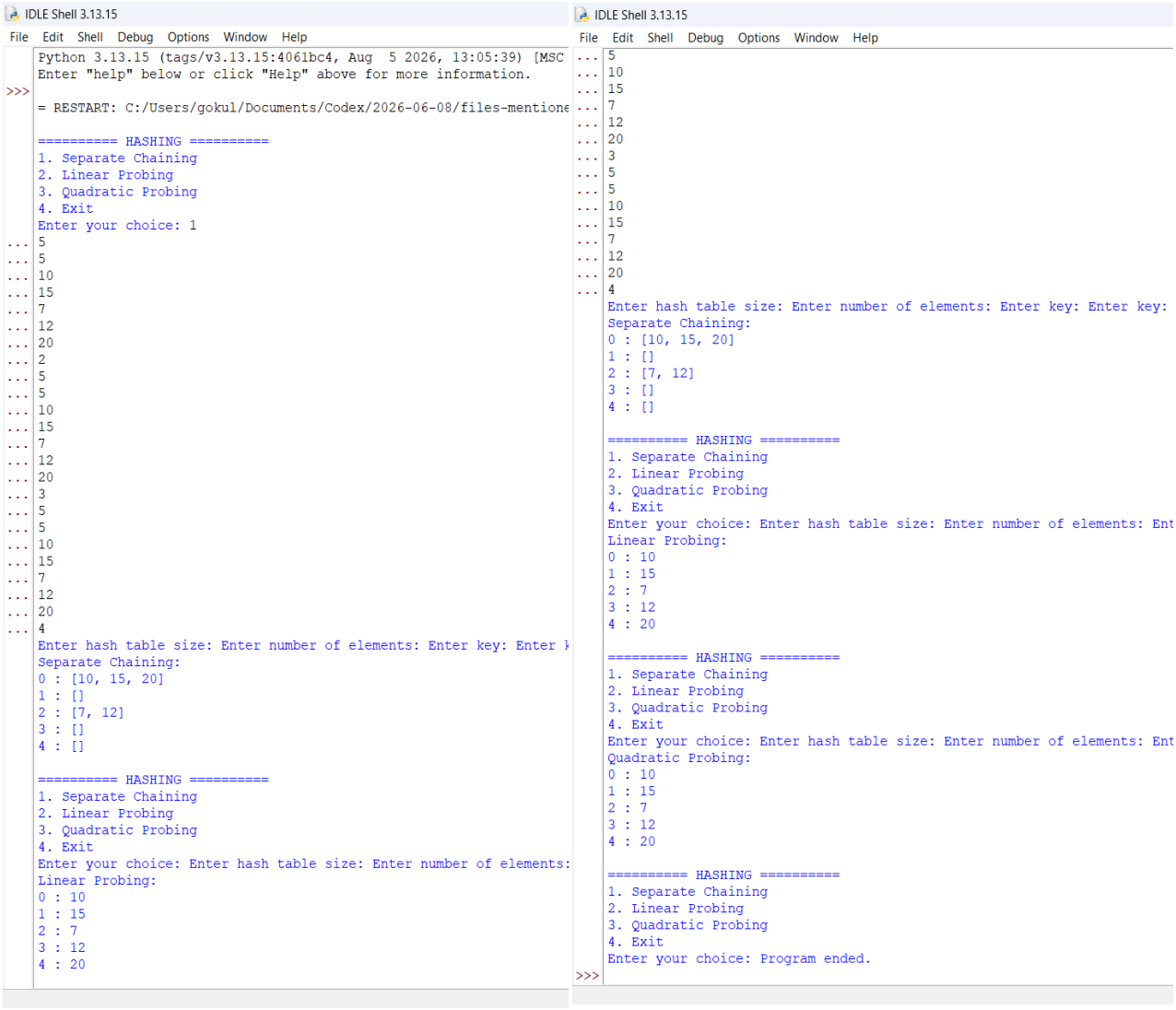
    elif choice == 3:
        quadratic_probing()

    elif choice == 4:
        print("Program ended.")
        break

    else:
```

print("Invalid choice!")

Output:



Explanation:

The program implements hashing using Separate Chaining, Linear Probing, and Quadratic Probing to handle collisions in a hash table. A hash function is used to calculate the index, and the appropriate collision resolution technique is applied to store the keys efficiently.

Error table:

S.No.	Error	Cause	Correction
1	Invalid choice	Wrong menu option entered	Enter a valid choice
2	Invalid input	Non-integer value entered	Enter integer values
3	Hash table is full	No empty position available	Increase the table size
4	Collision occurs	Two keys have the same hash index	Use chaining or probing technique